

**МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ**
**ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ
УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ**
**НОВОСИБИРСКИЙ НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ
ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ**
Факультет информационных технологий
Кафедра параллельных вычислений

ОТЧЕТ

О ВЫПОЛНЕНИИ ПРАКТИЧЕСКОЙ РАБОТЫ

«Умножение матрицы на матрицу в MPI 2D решетке»

студента 2 курса, 24203 группы

Анисимова Льва Евгеньевича

Направление 09.03.01 – «Информатика и вычислительная техника»

Преподаватель:
доцент кафедры параллельных
вычислений
Андрей Юрьевич Власенко

Новосибирск, 2026

СОДЕРЖАНИЕ

ЦЕЛЬ.....	3
ЗАДАНИЕ.....	3
ОПИСАНИЕ РАБОТЫ.....	4
Приложение 1. <i>Код последовательной программы</i>	7
Приложение 2. <i>Код программы tri.spp</i>	8
Приложение 3. <i>Код скрипта запуска на кластере</i>	12
Приложение 4. <i>Таблицы</i>	16
Приложение 5. <i>Графики</i>	17
Приложение 6. <i>Профилирование</i>	18

ЦЕЛЬ

Освоение концепции MPI-коммуникаторов и декартовых топологий, а также концепции производных типов данных в контексте параллельного умножения матриц.

ЗАДАНИЕ

1. Реализовать параллельный алгоритм умножения матрицы на матрицу при 2D решетке.
2. Исследовать производительность параллельно размера матрицы и размера решетки.
3. Выполнить профилирование программы с решетками 4x2 и 2x4 при помощи MPE на 8-и ядрах
4. Составить отчет.

ОПИСАНИЕ РАБОТЫ

В рамках выполнения лабораторной работы на языке C++ разработан и откомпилирован исходный код (Приложение 2), реализующий параллельный алгоритм умножения матриц $A[n_1 \times n_2]$ и $B[n_2 \times n_3]$ на двумерной решетке процессов $p_1 \times p_2$ с использованием технологии MPI. Экспериментальным путем подобран размер матриц (3120×2800 и 2800×3120), при котором время работы последовательной программы составляет ~ 171 секунду, что значительно превышает минимальный порог в 30 секунд, установленный заданием.

Параллельная реализация базируется на создании декартовой топологии процессов (*MPI_Cart_create*), что позволяет логически организовать вычислительные узлы в виде 2D-сетки. Для оптимизации коллективных операций внутри топологии выделены специализированные подкоммуникаторы строк и столбцов (*MPI_Cart_sub*).

Ключевые этапы реализации и механизмы взаимодействия процессов:

- **Декомпозиция данных:** Матрица A распределяется горизонтальными полосами по первому столбцу решетки процессов. Матрица B распределяется вертикальными полосами по первой строке решетки.

- **Использование производных типов:** Для передачи вертикальных полос матрицы B, данные которой в памяти расположены нелинейно, применен тип *MPI_Type_vector*. Это позволило инкапсулировать логику упаковки данных с шагом на уровне MPI-библиотеки.

- **Схема обмена данными:** Взаимодействие организовано по двухэтапной схеме. Сначала данные распределяются по краям решетки (комбинация *MPI_Scatter* и *MPI_Send*), после чего осуществляется рассылка полос A по строкам и полос B по столбцам подкоммуникаторов. Это обеспечивает наличие на каждом процессе (i, j) необходимых блоков для вычисления частного минора матрицы C.

- **Сбор результатов:** Результирующая матрица C формируется на процессе-корне (0,0) путем сбора миноров от всех процессов решетки. Для корректной сборки данных в нелинейные области общей матрицы на стороне получателя применен производный тип *MPI_Type_create_subarray*. Прием данных реализован через серию операций *MPI_Recv*, что позволило обойти ограничение идентичности буферов в стандартном *MPI_Gather*.

Проведены замеры времени выполнения программы для различных конфигураций решетки (2x12, 3x8, 4x6, 6x4, 8x3, 12x2) при фиксированном количестве процессов (24). Также исследована масштабируемость алгоритма при использовании от 1 до 24 вычислительных ядер.

На основании полученных данных выполнен расчет показателей ускорения S и эффективности распараллеливания E. Для анализа временных характеристик и выявления дисбаланса нагрузки проведено профилирование программы с использованием библиотеки MPE и визуализатора Jumpshot, зафиксировавшее высокую плотность вычислений и низкую долю накладных

расходов на коммуникации. Результаты измерений и графические зависимости приведены в соответствующих приложениях

ЗАКЛЮЧЕНИЕ

В ходе выполнения лабораторной работы была реализована параллельная версия алгоритма умножения матриц на базе двумерной решетки процессов с использованием технологий MPI и декартовых топологий. Экспериментальные данные подтвердили высокую масштабируемость алгоритма: при использовании 24 процессов достигнуто ускорение $S \approx 22.6$ при эффективности $E \approx 94\%$. Выбранный размер матриц (3120×2800 и 2800×3120) позволил минимизировать влияние латентности сети и продемонстрировать потенциал вычислительного узла.

Анализ производительности различных конфигураций решетки (от 2×12 до 12×2) показал инвариантность времени вычислений к форме топологии для данных объемов задач. Это свидетельствует о качественной реализации декомпозиции данных и сбалансированном распределении вычислительной нагрузки между узлами решетки. Использование производных типов данных (*MPI_Type_vector* и *MPI_Type_subarray*) позволило избежать явного копирования нелинейных участков памяти в промежуточные буферы, что сократило накладные расходы на этапе подготовки данных и их финальной сборки на процессе-корне.

Профилирование программы с помощью инструментов MPE и Jumpshot выявило, что основной вклад в общее время выполнения вносят непосредственно матричные вычисления, в то время как доля коммуникационных задержек остается пренебрежимо малой (менее 5% от общего времени). Это обусловлено высокой интенсивностью арифметических операций в алгоритме умножения матриц ($O(n^3)$ операций против $O(n^2)$ пересылок), что делает данный алгоритм классическим примером вычислительно-зависимой задачи, эффективно масштабируемой в распределенных системах.

На основании построенных графиков ускорения и эффективности можно сделать вывод о высокой пригодности разработанной реализации для работы на современных кластерных системах. В отличие от задач, ограниченных пропускной способностью памяти, данный алгоритм сохраняет линейный рост производительности вплоть до полной загрузки всех физических ядер, а использование $2D$ -декартовой топологии минимизирует объем передаваемых данных на каждом этапе по сравнению с одномерной декомпозицией.

Приложение 1. Код последовательной программы

```
#include <cstdlib>
#include <cstdio>
#include <ctime>

static double* alloc_zero(int n) { return new double[n](); }

static void matmul(const double* A, const double* B, double* C,
                  int n1, int n2, int n3)
{
    for (int i = 0; i < n1; ++i)
        for (int k = 0; k < n2; ++k) {
            double a = A[i * n2 + k];
            for (int j = 0; j < n3; ++j)
                C[i * n3 + j] += a * B[k * n3 + j];
        }
}

int main(int argc, char** argv)
{
    int n1 = argc > 1 ? atoi(argv[1]) : 1000;
    int n2 = argc > 2 ? atoi(argv[2]) : 1000;
    int n3 = argc > 3 ? atoi(argv[3]) : 1000;

    printf("Sequential: A[%d x %d] x B[%d x %d] = C[%d x %d]\n",
          n1, n2, n2, n3, n1, n3);

    double* A = alloc_zero(n1 * n2);
    double* B = alloc_zero(n2 * n3);
    double* C = alloc_zero(n1 * n3);

    srand(42);
    for (int i = 0; i < n1 * n2; ++i) A[i] = (double)(rand() % 10);
    for (int i = 0; i < n2 * n3; ++i) B[i] = (double)(rand() % 10);

    struct timespec ts, te;
    clock_gettime(CLOCK_MONOTONIC, &ts);
    matmul(A, B, C, n1, n2, n3);
    clock_gettime(CLOCK_MONOTONIC, &te);

    double elapsed = (te.tv_sec - ts.tv_sec) + (te.tv_nsec -
ts.tv_nsec) * 1e-9;
    printf("t_sequential = %.4f s\n", elapsed);

    delete[] A; delete[] B; delete[] C;
    return 0;
}
```

Приложение 2. Код программы *tri.crr*

```
#include <mpi.h>
#include <cstdlib>
#include <cstring>
#include <cmath>
#include <stdio>

static double* alloc_zero(int n) { return new double[n](); }

// C += A*B, A:[ra×k], B:[k×cb], C:[ra×cb]
static void matmul(const double* A, const double* B, double* C,
                  int ra, int k, int cb)
{
    for (int i = 0; i < ra; ++i)
        for (int l = 0; l < k; ++l) {
            double a = A[i * k + l];
            for (int j = 0; j < cb; ++j)
                C[i * cb + j] += a * B[l * cb + j];
        }
}

int main(int argc, char** argv)
{
    MPI_Init(&argc, &argv);
    int rank, size;
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &size);

    int p1, p2;
    if (argc >= 3) { p1 = atoi(argv[1]); p2 = atoi(argv[2]); }
    else {
        p1 = (int)sqrt(size);
        while (size % p1) --p1;
        p2 = size / p1;
    }
    if (p1 * p2 != size) {
        if (!rank) fprintf(stderr, "Ошибка: p1*p2 != size (%d*d != %d)\n", p1, p2, size);
        MPI_Finalize(); return 1;
    }

    /*      —      размеры      матриц
    */

    int n1, n2, n3;
    if (argc >= 6) {
        n1 = atoi(argv[3]); n2 = atoi(argv[4]); n3 = atoi(argv[5]);
    } else {
        n1 = n2 = n3 = 2400;
    }

    if (n1 % p1 != 0 || n3 % p2 != 0) {
        if (!rank)
            fprintf(stderr, "Ошибка: n1(%d) %% p1(%d) = %d, n3(%d) %% p2(%d) = %d\n",
                    n1, p1, n1%p1, n3, p2, n3%p2);
        MPI_Finalize(); return 1;
    }
}
```

```

const int rows_a = n1 / p1;    // строк A у каждого процесса
const int cols_b = n3 / p2;    // столбцов B у каждого процесса

if (!rank)
    printf("Grid %dx%d | A[%dx%d] B[%dx%d] C[%dx%d] | rows_a=%d\n",
           p1, p2, n1, n2, n3, n1, n3, rows_a, cols_b);

/*      —      декартова      топология */

int dims[2] = {p1, p2};
int periods[2] = {0, 0};
MPI_Comm cart;
MPI_Cart_create(MPI_COMM_WORLD, 2, dims, periods, 0, &cart);

int coords[2];
MPI_Cart_coords(cart, rank, 2, coords);
const int my_row = coords[0], my_col = coords[1];

int rem_row[2] = {0, 1}; // все процессы одной строки решетки
MPI_Comm row_comm;
MPI_Cart_sub(cart, rem_row, &row_comm);

int rem_col[2] = {1, 0}; // все процессы одного столбца решетки
MPI_Comm col_comm;
MPI_Cart_sub(cart, rem_col, &col_comm);

/*      —      буферы */

double *A_full = 0, *B_full = 0, *C_full = 0;
double* la = alloc_zero(rows_a * n2); // локальная полоса A
double* lb = alloc_zero(n2 * cols_b); // локальная полоса B
double* lc = alloc_zero(rows_a * cols_b); // локальный минор C

/*      —      генерация матриц на процессе (0,0) */

if (!rank) {
    A_full = alloc_zero(n1 * n2);
    B_full = alloc_zero(n2 * n3);
    C_full = alloc_zero(n1 * n3);
    srand(42);
    for (int i = 0; i < n1*n2; ++i) A_full[i] = (double)(rand() %
10);
    for (int i = 0; i < n2*n3; ++i) B_full[i] = (double)(rand() %
10);
}

/* =====
 * ШАГ 3: Scatter горизонтальных полос A по col_comm
 * ===== */
if (my_col == 0)
    MPI_Scatter(A_full, rows_a*n2, MPI_DOUBLE,
               la, rows_a*n2, MPI_DOUBLE,
               0, col_comm);

/* =====
 * ШАГ 4: Раздача вертикальных полос B по строке 0 (point-to-

```

```

point)
    * ===== */
    if (my_row == 0) {
        if (my_col == 0) {
            for (int r = 0; r < n2; ++r)
                memcpy(lb + r*cols_b, B_full + r*n3, cols_b *
sizeof(double));

            for (int j = 1; j < p2; ++j) {
                int dest; int dc[2] = {0, j};
                MPI_Cart_rank(cart, dc, &dest);

                MPI_Datatype vtype; // вертикальная полоса j
                MPI_Type_vector(n2, cols_b, n3, MPI_DOUBLE, &vtype);
                MPI_Type_commit(&vtype);
                MPI_Send(B_full + j*cols_b, 1, vtype, dest, 100,
cart);
                MPI_Type_free(&vtype);
            }
        } else {
            int src; int sc[2] = {0, 0};
            MPI_Cart_rank(cart, sc, &src);
            MPI_Recv(lb, n2*cols_b, MPI_DOUBLE, src, 100, cart,
MPI_STATUS_IGNORE);
        }
    }

    /* =====
    * ШАГ 5: Broadcast полос A по строкам
    * ===== */
    MPI_Bcast(la, rows_a*n2, MPI_DOUBLE, 0, row_comm);

    /* =====
    * ШАГ 6: Broadcast полос B по столбцам
    * ===== */
    MPI_Bcast(lb, n2*cols_b, MPI_DOUBLE, 0, col_comm);

    /* =====
    * ШАГ 7: Локальное умножение
    * ===== */
    MPI_Barrier(cart);
    double t0 = MPI_Wtime();
    matmul(la, lb, lc, rows_a, n2, cols_b);
    double t_compute = MPI_Wtime() - t0;

    /* =====
    * ШАГ 8: Сбор миноров C на (0,0)
    * ===== */
    if (rank == 0) {
        for (int r = 0; r < rows_a; ++r)
            memcpy(C_full + r*n3, lc + r*cols_b, cols_b *
sizeof(double));

        for (int i = 0; i < p1; ++i) {
            for (int j = 0; j < p2; ++j) {
                if (i == 0 && j == 0) continue;
                int src; int sc[2] = {i, j};
                MPI_Cart_rank(cart, sc, &src);

```

```

        int arr_sz[2] = {n1, n3};
        int sub_sz[2] = {rows_a, cols_b};
        int starts[2] = {i*rows_a, j*cols_b};
        MPI_Datatype sub_t;
        MPI_Type_create_subarray(2, arr_sz, sub_sz, starts,
                                MPI_ORDER_C, MPI_DOUBLE,
&sub_t);

        MPI_Type_commit(&sub_t);
        MPI_Recv(C_full, 1, sub_t, src, 200, cart,
MPI_STATUS_IGNORE);
        MPI_Type_free(&sub_t);
    }
} else {
    int root; int rc[2] = {0, 0};
    MPI_Cart_rank(cart, rc, &root);
    MPI_Send(lc, rows_a*cols_b, MPI_DOUBLE, root, 200, cart);
}

MPI_Barrier(cart);
double t_total = MPI_Wtime() - t0;

/* — Верификация ————— */
if (rank == 0) {
    if (n1 <= 300 && n2 <= 300 && n3 <= 300) {
        double* C_ref = alloc_zero(n1*n3);
        matmul(A_full, B_full, C_ref, n1, n2, n3);
        double mx = 0;
        for (int i = 0; i < n1*n3; ++i) mx = fmax(mx,
fabs(C_full[i]-C_ref[i]));
        printf("Верификация: max_err=%.2e  %s\n", mx, mx < 1e-6 ?
"OK" : "FAIL");
        delete[] C_ref;
    }
    printf("t_compute = %.4f s\nt_total    = %.4f s\n", t_compute,
t_total);
}

/* ————— Освобождение */

delete[] la; delete[] lb; delete[] lc;
if (rank == 0) { delete[] A_full; delete[] B_full; delete[]
C_full; }

MPI_Comm_free(&row_comm);
MPI_Comm_free(&col_comm);
MPI_Comm_free(&cart);
MPI_Finalize();
return 0;
}

```

Приложение 3. Код скрипта запуска на кластере

```
#!/bin/bash
#PBS -N MatMul_MPI_2D
#PBS -l nodes=2:ppn=12
#PBS -l walltime=03:00:00
#PBS -j oe

cd $PBS_O_WORKDIR

#
=====
# Компиляция (-O1 чтобы последовательная версия давала ~30 сек)
#
=====
echo "=== Компиляция ==="
mpicc -xcrarch=ppc ./src/mpi_prog.cpp -o mpi_matmul -O0 -std=c++11
mpicc -xcrarch=ppc ./src/sequential.cpp -o sequential -O0
mpicc -xcrarch=ppc -xcrarch=ppc ./src/mpi_prog.cpp -o mpi_matmul_mpe -O0 -std=c++11
echo "Компиляция завершена."

# — Размеры матриц
=====
# Квадратная матрица N×N×N (ТЗ: квадратная, но программа работает с любыми).
# N должно делиться на все p1, p2 из {1,2,3,4,6,8,12} → lcm=24 → N=2400
# n2=2800 (n2 – внутренняя размерность, делимость не нужна):
# при -O1 даёт последовательной ~30 сек.
N=3120
N2=2800

#
=====
# БЛОК 1: Последовательный запуск (квадратная матрица N×N2×N)
#
=====
echo ""
echo "===== "
echo " БЛОК 1: Последовательный запуск"
echo " A[${N}x${N2}] x B[${N2}x${N}] (квадратная C[${N}x${N}]) "
echo "===== "

./sequential $N $N2 $N | tee sequential.txt

T_SEQ=$(grep "t_sequential" sequential.txt | awk '{print $3}')
echo "Последовательное время: ${T_SEQ} с"

#
=====
# БЛОК 2: Влияние формы решетки при 24 процессах (квадратная матрица)
#
# Все решетки произведения = 24, матрица одна и та же → видим как
# форма решетки (соотношение p1:p2) влияет на время.
# Квадратная решетка (≈4x6) ожидаемо даёт минимум времени.
#
=====
echo ""
echo "===== "
echo " БЛОК 2: Влияние формы решетки (24 процесса) "
```

```

echo "  A[${N}x${N2}] x B[${N2}x${N}]"
echo "===== "

declare -A GRIDS
GRIDS["2x12"]="2 12"
GRIDS["3x8"]="3 8"
GRIDS["4x6"]="4 6"
GRIDS["6x4"]="6 4"
GRIDS["8x3"]="8 3"
GRIDS["12x2"]="12 2"

for GNAME in "2x12" "3x8" "4x6" "6x4" "8x3" "12x2"; do
    read P1 P2 <<< "${GRIDS[$GNAME]}"
    echo ""
    echo "--- Решетка ${GNAME} ---"
    mpirun -machinefile $PBS_NODEFILE -np 24 -perhost 12 \
        ./mpi_matmul $P1 $P2 $N $N2 $N \
        > result_grid_${GNAME}.txt 2>&1
    cat result_grid_${GNAME}.txt
done

echo ""
echo "--- Сводная таблица: влияние формы решетки (24 процесса) ---"
printf "%-8s | %-13s | %-12s\n" "Решетка" "t_compute (s)" "t_total (s)"
printf -- "-----|-----|-----\n"
for GNAME in "2x12" "3x8" "4x6" "6x4" "8x3" "12x2"; do
    TC=$(grep "t_compute" result_grid_${GNAME}.txt | awk '{print $3}')
    TT=$(grep "t_total" result_grid_${GNAME}.txt | awk '{print $3}')
    printf "%-8s | %-13s | %-12s\n" "$GNAME" "${TC:--}" "${TT:--}"
done

#
=====
#
# БЛОК 3: Ускорение и эффективность от числа процессов
#
# Используем наиболее квадратную решетку для каждого NP.
# Ожидаемый результат - "график корыта":
#   - Ускорение  $S = T_{seq} / T_{parallel}$  растёт, но медленнее NP
#   - Эффективность  $E = S / NP$  монотонно падает с ростом NP
#   - Видно как накладные расходы (коммуникации) "съедают" прирост
#
=====
#
echo ""
echo "===== "
echo "  БЛОК 3: Ускорение и эффективность"
echo "  A[${N}x${N2}] x B[${N2}x${N}]"
echo "===== "

# Формат: "NP P1 P2" - наиболее квадратная решетка для каждого NP
CONFIGS=(
    "1 1 1"
    "2 1 2"
    "4 2 2"
    "6 2 3"
    "8 2 4"
    "12 3 4"
    "16 4 4"
    "24 4 6"
)

for CFG in "${CONFIGS[@]"; do
    read NP P1 P2 <<< "$CFG"

```

```

echo ""
echo "--- ${NP} процессов, решетка ${P1}x${P2} ---"
if [ "$NP" -eq 1 ]; then
    mpirun -machinefile $PBS_NODEFILE -np 1 \
        ./mpi_matmul 1 1 $N $N2 $N \
        > result_np${NP}.txt 2>&1
elif [ "$NP" -le 12 ]; then
    mpirun -machinefile $PBS_NODEFILE -np $NP \
        ./mpi_matmul $P1 $P2 $N $N2 $N \
        > result_np${NP}.txt 2>&1
else
    mpirun -machinefile $PBS_NODEFILE -np $NP -perhost 12 \
        ./mpi_matmul $P1 $P2 $N $N2 $N \
        > result_np${NP}.txt 2>&1
fi
cat result_np${NP}.txt
done

echo ""
echo "--- Таблица ускорения и эффективности ---"
printf "%-4s | %-8s | %-13s | %-12s | %-11s | %-11s\n" \
    "NP" "Решетка" "t_compute (s)" "t_total (s)" "Ускорение S" "Эффект. E"
printf --
"-----|-----|-----|-----|-----|-----"
\n"

for CFG in "${CONFIGS[@]"; do
    read NP P1 P2 <<< "$CFG"
    TC=$(grep "t_compute" result_np${NP}.txt | awk '{print $3}')
    TT=$(grep "t_total" result_np${NP}.txt | awk '{print $3}')
    if [ -n "$TC" ] && [ -n "$T_SEQ" ]; then
        S=$(awk "BEGIN{printf \"%.4f\", ${T_SEQ}/${TC}}")
        E=$(awk "BEGIN{printf \"%.4f\", ${S}/${NP}}")
    else
        S="-"; E="-"
    fi
    printf "%-4s | %-8s | %-13s | %-12s | %-11s | %-11s\n" \
        "$NP" "${P1}x${P2}" "${TC:--}" "${TT:--}" "$S" "$E"
done

#
=====
# БЛОК 4: Профилирование с MPE (-mpilog), 8 процессов
#
=====
echo ""
echo "=====
echo " БЛОК 4: Профилирование с MPE (-mpilog), 8 процессов"
echo " A[${N}x${N2}] x B[${N2}x${N}]"
echo " → генерируется .clog2 для jumpshot"
echo "=====

for GNAME in "2x4" "4x2"; do
    P1=$(echo $GNAME | cut -d'x' -f1)
    P2=$(echo $GNAME | cut -d'x' -f2)
    echo ""
    echo "--- Профилирование решетка ${GNAME} ---"
    mpirun -machinefile $PBS_NODEFILE -np 8 \
        ./mpi_matmul_mpe $P1 $P2 $N $N2 $N \
        > profile_${GNAME}.txt 2>&1
    cat profile_${GNAME}.txt

```

```

TC=$(grep "t_compute" profile_${GNAME}.txt | awk '{print $3}')
if [ -n "$TC" ] && [ -n "$T_SEQ" ]; then
    S=$(awk "BEGIN{printf \"%.4f\\\", ${T_SEQ}/${TC}}")
    E=$(awk "BEGIN{printf \"%.4f\\\", ${S}/8}")
    echo "    Ускорение S = ${S},    Эффективность E = ${E}"
fi

if [ -f mpi_matmul_mpe.clog2 ]; then
    mv mpi_matmul_mpe.clog2 profile_${GNAME}.clog2
    echo "    Лог: profile_${GNAME}.clog2"
fi
done

echo ""
echo "=== Готово ==="
echo "Просмотр: jumpshot profile_2x4.clog2"

```

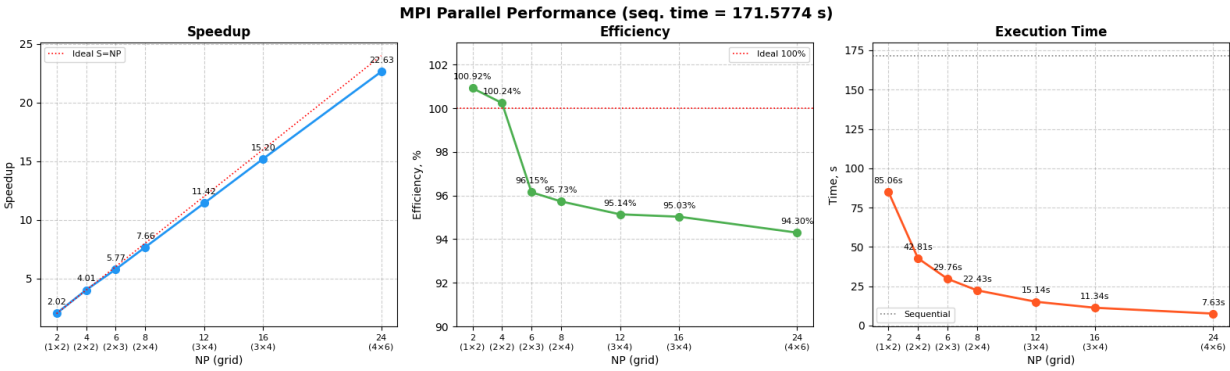
Приложение 4. Таблицы

NP	РЕШЕТКА	ВРЕМЯ	УСКОРЕНИЕ	ЭФФЕКТИВНОСТЬ	ВРЕМЯ ПОСЛЕДОВАТЕЛЬНОЙ: 171.5774
2	1x2	85.0630	2.0185	100.92%	
4	2x2	42.8131	4.0094	100.24%	
6	2x3	29.7635	5.7689	96.15%	
8	2x4	22.4269	7.6586	95.73%	
12	3x4	15.1424	11.4168	95.14%	
16	3x4	11.3352	15.2043	95.03%	
24	4x6	7.6299	22.6320	94.30%	

РЕШЕТКА *	ВРЕМЯ РАСЧЕТА	ВРЕМЯ ИСПОЛНЕНИЯ
2x12	7.5861	7.6267
3x8	7.5782	7.6270
4x6	7.5792	7.6500
6x4	7.5586	7.6234
8x3	7.5527	7.6103
12x2	7.5499	7.6122

**Замеры сделаны при NP=24.*

Приложение 5. Графики



Приложение 6. Профилирование

